Análise de Eficácia de Testes com Teste de Mutação

Relatório de atividade prática

Disciplina: Teste de Software

Aluno: João Vitor Pedersoli Rajão

Matrícula: não informada

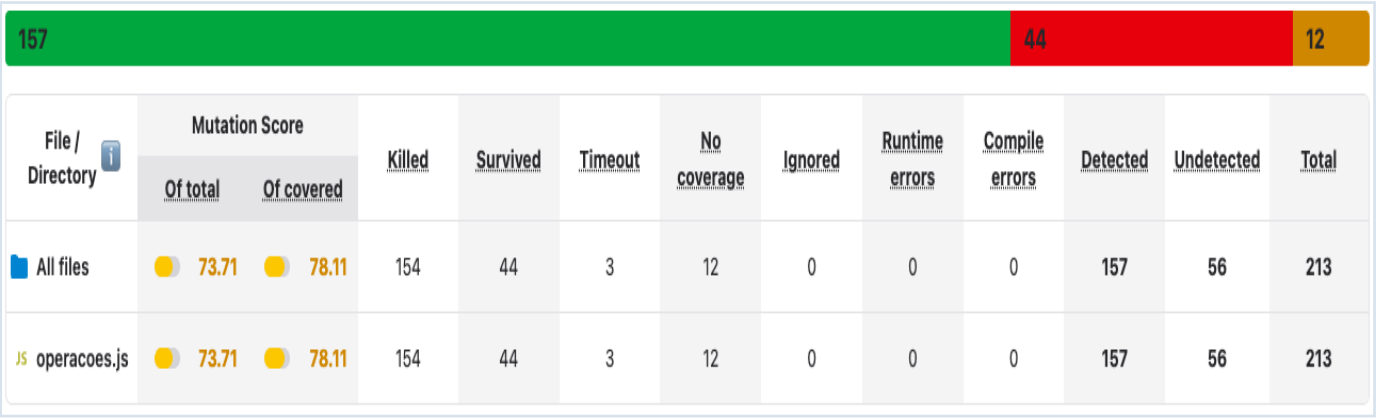
1. Análise inicial

Comecei pela suíte que veio no repositório: 50 testes Jest para 50 funções. Todos passaram na primeira execução.

Linhas cobertas	Ramos cobertos	Mutação	Sobreviventes
98,64%	58,82%	73,71%	44

A cobertura de linhas parecia boa. Ao rodar o Stryker, porém, encontrei 44 mutantes sobreviventes e 12 sem cobertura, entre 213 gerados. Os testes passavam por quase todas as linhas, mas deixavam de fora casos simples: array vazio, zero, números compostos e valores iguais nas comparações.

Evidência - primeira execução do StrykerJS

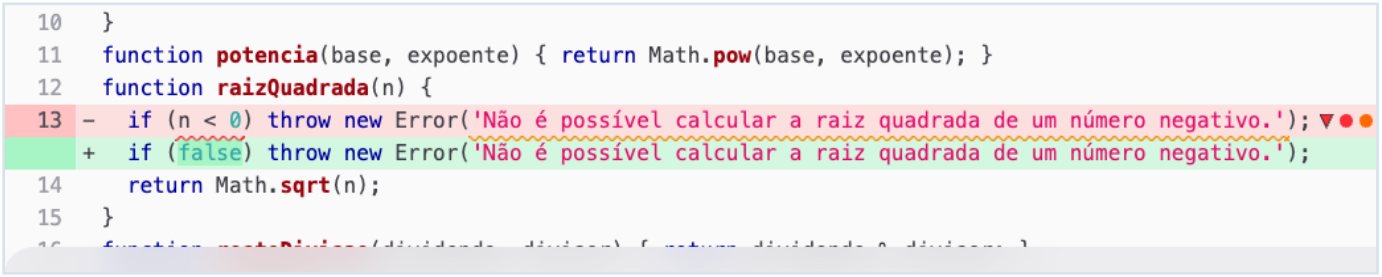


Na medição inicial, 154 mutantes foram mortos e 3 causaram timeout. A cobertura de statements era 85,41% e a de funções, 100%. Chamar uma função no teste claramente não bastava para conferir seus diferentes resultados.

Repositório: github.com/JoaoRajao/operacoes-mutante

2. Três falhas reveladas pelo relatório

A. Raiz quadrada - guarda removida



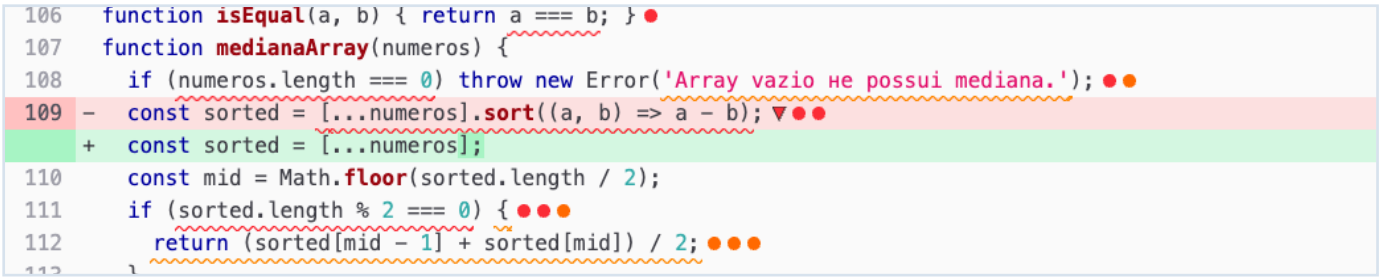
O Stryker trocou if (n < 0) por if (false). Como o teste só usava 16, continuou passando. Acrescentei raizQuadrada(-4) com a mensagem esperada e testei 0 como limite válido. Com -4, a mutação é detectada.

B. Primalidade - rejeição de composto removida



O teste de isPrimo usava apenas 7. Ao substituir a condição de divisibilidade por false, ainda recebia true. Acrescentei 4 e 9, que devem retornar false, além de 0, 1 e 2 para verificar os limites da função.

C. Mediana - ordenação eliminada



A lista do teste original já estava ordenada. Por isso, retirar o sort não mudava a resposta. Usei [9, 1, 5] e [8, 1, 4, 2]: sem ordenar, a mediana fica errada. O segundo caso também testa o cálculo para lista par.

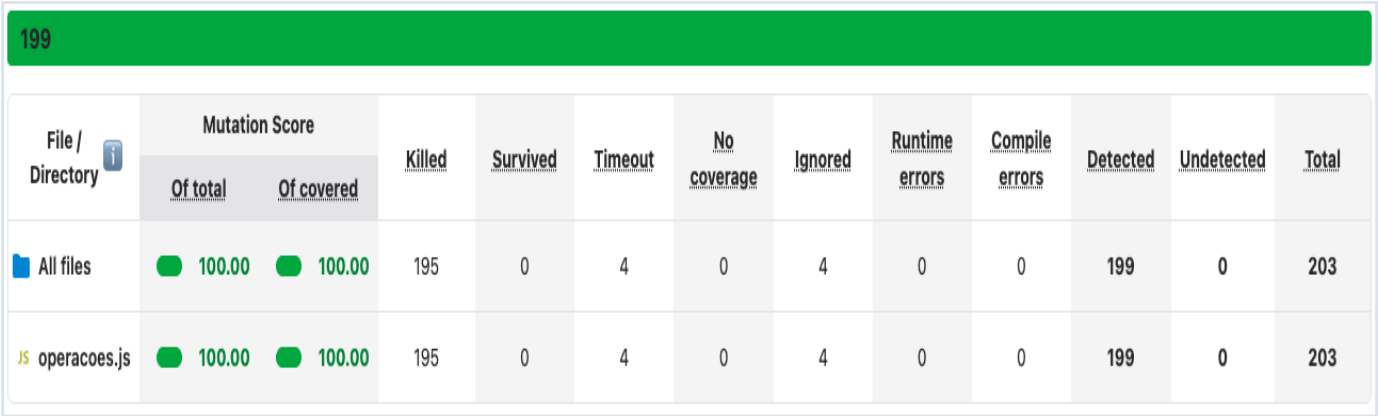
Solução, resultados e conclusão

3. Suíte aprimorada

Acrescentei 14 testes, passando de 50 para 64. Além dos casos da página anterior, testei arrays vazios, erros, paridade, limites do clamp, comparações com resultado falso e conversões de temperatura fora de zero. Os 64 testes passam; a cobertura final de linhas e de ramos chegou a 100%.

4. Pontuação final e tratamento dos equivalentes

Com os novos testes e o código original, a pontuação foi de 73,71% para 96,71%: 203 mutantes mortos, 3 timeouts e 7 sobreviventes. Examinei esses sete. Cinco vinham de condições redundantes em fatorial e produtoArray; dois trocavam < por <= ou > por >= nas bordas do clamp. Nessas entradas, o retorno não muda. Removi os ramos redundantes e justifiquei no código quatro mutantes de EqualityOperator marcados como Ignored pelo Stryker.



199

File / Directory	Mutation Score		Killed	Survived	Timeout	No coverage	Ignored	Runtime errors	Compile errors	Detected	Undetected	Total
	Of total	Of covered										
All files	100.00	100.00	195	0	4	0	4	0	0	199	0	203
operacoes.js	100.00	100.00	195	0	4	0	4	0	0	199	0	203

Na última execução, o Stryker mostrou 100,00%: 195 mortos, 4 timeouts, 4 ignorados e nenhum sobrevivente. Esse 100% considera as exclusões e a remoção dos ramos redundantes. O ganho causado somente pelos novos testes foi de 23,00 pontos percentuais, de 73,71% para 96,71%.

5. Conclusão

A maior diferença veio de testar entradas que eu ainda não tinha considerado, não de chamar mais funções. A cobertura de linhas já era alta; os mutantes mostraram onde faltava conferir o resultado. Também precisei separar falhas reais de mutações que não alteram o comportamento da função.

Evidências: evidencias/inicial, evidencias/sem-exclusoes e evidencias/final.

Fonte: StrykerJS - stryker-mutator.io/docs/stryker-js/disable-mutants/

Código e relatório: github.com/JoaoRajao/operacoes-mutante